COMP 421 - Database Systems
Assignment 3: MapReduce and Pig Latin

Seyed Yazdan Zinati
seyed.zinati@mail.mcgill.ca
260928077

April 3, 2022

Q1

a

Mapper:
For each input key-value pair (linenumber, linetext):
    Output (extract_browser(linetext), extract_IP(linetext))

Group and Shuffle will create for each value of browser (extracted using helper function), a key-value list: (Browser, (IP_1, IP_2, ...))

Reducer:
For each (Browser (IP_1, IP_2, ...)):
    result = (browser, count of distinct IP in (IP_1, IP_2, ...) as num_users)
    if num_users > 100, output result

b

Mapper input:

Line 1: 1.1.2 - frank [10/Oct/2000:13:55:36 -0700] "GET /apache pb.gif HTTP/1.0" 200 2326"http://www.example.com/start.html" "Mozilla/4.08"

Line 2: 1.1.2 - frank [10/Oct/2000:12:55:36 -0700] "GET /apache pb.gif HTTP/1.0" 200 2326"http://www.example.com/start.html" "Mozilla/4.08"

Line 3: 1.0.0 - anon [10/Apr/2022:00:55:36 -0700] "GET /apache pb.gif HTTP/1.0" 200 2326"http://www.example.com/start.html" Chromium"

Line 4: 1.0.3 - sophie [10/Apr/2022:00:55:36 -0700] "GET /apache pb.gif HTTP/1.0" 200 2326"http://www.example.com/start.html" Chromium"

Line 4: 1.0.4 - rose [10/Apr/2022:00:55:36 -0700] "GET /apache pb.gif HTTP/1.0" 200 2326"http://www.example.com/start.html" Mozilla/4.08

...

Line N: ...

Note: IPV4 addresses were shortened for simplicity

Mapper output: (Mozilla/4.08, 1.1.2), (Mozilla/4.08, 1.1.2), (Chromium, 1.0.0), (Chromium, 1.0.3), (Mozilla/4.08, 1.0.4), ...

Reducer input: (Mozilla/4.08, (1.1.2, 1.1.2, 1.1.4, ...)), (Chromium, (1.0.0, 1.0.3, ...))

Reducer output: Assume that firefox (Mozilla) had less than 100 unique IP addresses in this step and Chromium had 1000 unique IP addresses, then the output will be: (Chromium, 1000). We note that there is a duplicate IP in the Mozilla browser tuple which will not be double-counted since we count distinct values.

c

We can use combine functionality for reducing the network usage and IO. At each mapper, we could perform the same logic as the reducer (counting the number of unique IP addresses). However, contrary to the reducer, we will not filter tuples with less than 100 unique IP addresses as more IP addresses could be added during grouping and shuffling from other mappers. This will reduce network and IO cost since less data will be read and transferred.

Q2

a

We need two pairs of Map/Reduce functions.

Mapper 1:
For each key-value pair (linenumber, linetext):
Output (extract_IP(linetext), extract_browser(linetext))

Group and Shuffle will create for each value of IP (representing users) extracted using provided helper functions a key-value list: (IP, ($Browser_1$, $Browser_2$, ...)).

Reducer 1:
For each (IP, ($Browser_1$, $Browser_2$, ...)):
Output (IP, 1) if count of of distinct values in ($Browser_1$, $Browser_2$, ...) is greater than 1.

Mapper 2:
For each key-value pair (IP, 1):
output (1, IP).

Group and Shuffle will create for each value of 1 (representing users that use multiple broswers) a key-value list: (1, (IP_1, IP_2, ...)).

Reducer 2:
For each (1, (IP_1, IP_2, ...)):
Output count of distinct values in list (IP_1, IP_2, ...).

See example of this workflow in Figure 1

Map 1 input	Map 1 output	Reduce 1 input	Reduce 1 output/ Mapper 2 input	Mapper 2 output	Reduce 2 input	Reduce 2 output
(1, text1)	(1, F)	(1, (F))	(2, 1)	(1, 2)	(1, (2, 4, 6))	(3)
(2, text2)	(2, F)	(2, (F, C))	(4, 1)	(1, 4)		
(3, text3)	(2, C)	(3, (C))	(6, 1)	(1, 6)		
(4, text4)	(3, C)	(4, (C, F, F))				
(5, text5)	(4, C)	(5, (S))				
(6, text6)	(4, F)	(6, (S, F, C, C))				
(7, text7)	(4, F)					
(8, text8)	(5, S)					
(9, text9)	(6, S)					
(10, text10)	(6, F)					
(11, text11)	(6, C)					
(12, text12)	(6, C)					



assuming the text results into this
after extracting IP and Browser fields
(We are short IPs of 1, 2, 3 and abbreviated Browsers)

S = Safari
F = Firefox
C = Chrome

Figure 1: Q2(a) example

b

As can be seen from Figure 1, the designed Map/Reduce workflow only requires 1 reducer process at the second reducer. This is because all tuples have the same key of 1 at the output of the second mapper. As a result, all tuples are grouped into 1 tuple of the following format : $(1, (IP_1, IP_2, ...))$. So in this special case, only one reducer process is needed.

Q6

What does the schema look like immediately after you perform the GROUP operation step?
Each group contains at least one tuple containing the following attributes: country, population, persons fully vaccinated, iso3, iso3, and product name.

```
grunt> DESCRIBE Grpd;
Grpd: {
    group: chararray,
    Jnd2: {
        (
            Proj::highpop::country: chararray,
            Proj::highpop::population: int,
            Proj::vdatacsv::persons_fully_vaccinated: int,
            Proj::vdatacsv::iso3: chararray,
            vmetacsv::iso3: chararray,
            vmetacsv::product_name: chararray
        )
    }
```

```
14 }
```